운영체제

5주차 – 프로세스 동기화

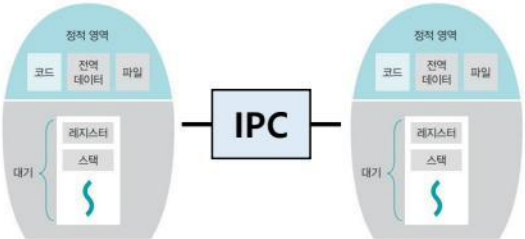
학습 목표

- 프로세스 간 동기화(IPC) 를 이해한다.
- 임계구역을 이해한다.

IPC

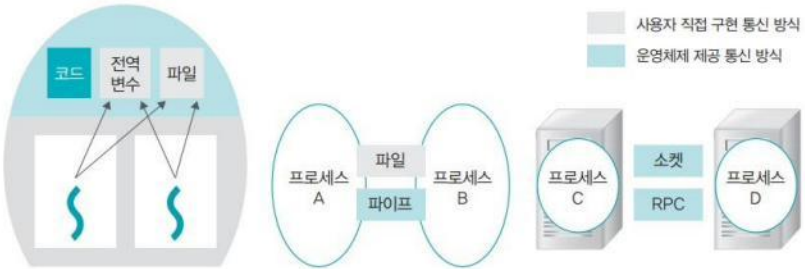
프로세스 간 통신의 개념 (1/2)

- **정의**
  - IPC : Inter-Process Communication
  - 프로세스간 데이터를 공유하기 위한 방법
  - 운영체제는 프로세스의 관리 및 각각의 동작을 보호
    - 프로세스는 자신의 할당메모리 영역 외에는 접근 불가(독립성)
    - 타프로세스의 영역을 독단적으로 접근할 수 없음
    - 하나의 프로세스가 오류가 나도 다른 프로세스에 영향을 주지 않음
  - 프로세스간 통신을 위해 운영체제는 몇가지 자원을 제공
    - 예: 시그널, 인터럽트, 공유 파일, 파이프, 공유메모리, 메시지큐 등



프로세스 간 통신의 개념 (2/2)

- **프로세스간 통신(IPC)**
  - 프로세스 내부 데이터 통신
    - 프로세스 내 스레드간 통신(스레드 ↔ 스레드)
      - 스레드는 **전역 변수**나 **파일**을 이용하여 데이터를 공유
      - But, 동기화(임계구역) 문제 발생



(a) 프로세스 내부 데이터 통신 (b) 프로세스 간 데이터 통신 (c) 네트워크를 이용한 데이터 통신

  - 프로세스 간 데이터 통신
    - 같은 컴퓨터(동일 호스트)에 있는 프로세스간 통신(프로세스 ↔ 프로세스)
    - **공용 파일** 또는 운영체제가 제공하는 **자원**을 이용
    - 파이프, 공유메모리, 메시지 큐 등

  - 네트워크를 이용한 데이터 통신
    - 여러 컴퓨터가 네트워크로 연결되어 있을 때 통신(컴퓨터 ↔ 컴퓨터)
    - 소켓을 이용한 데이터 공유

방향성에 따른 분류

- **단방향 통신(simplex communication)**
  - 한쪽 방향으로만 데이터를 전송할 수 있는 구조
  - 대표적 예: 파이프(PIPE) 기법이 있음

- **양방향 통신(duplex communication)**
  - 데이터를 동시에 양쪽 방향으로 전송할 수 있는 구조
  - 대표적 예: 소켓 통신

- **반양방향 통신(half-duplex communication)**
  - 양쪽 방향으로 전송이지만 동시 전송은 불가
  - 특정 시점에 한쪽 방향으로만 전송할 수 있는 구조
  - 대표적 예: 메시지 큐, 공유메모리

동시성에 따른 분류

- **대기가 있는 통신**
  - 동기화를 지원하는 통신 방식
  - 데이터를 받는 쪽은 데이터가 **도착할 때까지 자동 대기 상태(블록킹, Blocking)**

- **대기가 없는 통신**
  - 동기화를 지원하지 않는 통신 방식
  - 다른 작업을 하는 중에 데이터가 수신되면 처리
    - 인터럽트(Interrupt, 이벤트) 처리 방식
    - 폴링(Polling) 처리 방식

구분	블로킹 (Blocking)	논블로킹 (Non-Blocking)
별칭	동기(Synchronous)	비동기(Asynchronous)
동작	데이터 올 때까지 **대기**	데이터 무관하게 **계속 실행**
처리 방식	자동 대기	인터럽트 or 폴링
장점	구현 단순, 순서 보장	CPU 낭비 없음
단점	대기 중 CPU 낭비	구현 복잡

IPC 기법 (1/9)

- **종류**

기법	방향성	동시성	속도	사용 범위	특징
시그널	단방향	비동기	빠름	동일 호스트	제어 목적, 데이터 전달 ✕
인터럽트	단방향	비동기	매우 빠름	동일 호스트	이벤트 기반
공유 파일	양방향	동기	느림	동일 호스트	디스크 I/O 속도 종속
파이프	단방향	동기	보통	동일 호스트	부모-자식 간 주로 사용
메시지큐	반양방향	비동기	보통	동일 호스트	큐 방식, 부모-자식 무관
공유메모리	반양방향	비동기	**가장 빠름**	동일 호스트	동기화 필수
소켓	양방향	동기/비동기	보통	**네트워크**	원격 통신 가능

IPC 기법 (2/9)

- **시그널(Signal)**
  - 운영체제는 프로세스를 관리하기 위한 시그널을 정의
  - 시그널을 수신한 프로세스는 그에 맞는 행위를 수행
    - 예1 : 9(SIGKILL)는 프로세스를 강제 종료
    - 예2 : 14(SIGALRM)은 잠들어있는 프로세스를 깨움

```
[kali@kali:~]$kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
[kali@kali:~]$
```

① sleep 명령 실행

```
[kali@kali:~]$sleep 100
Alarm clock
[kali@kali:~]$
```

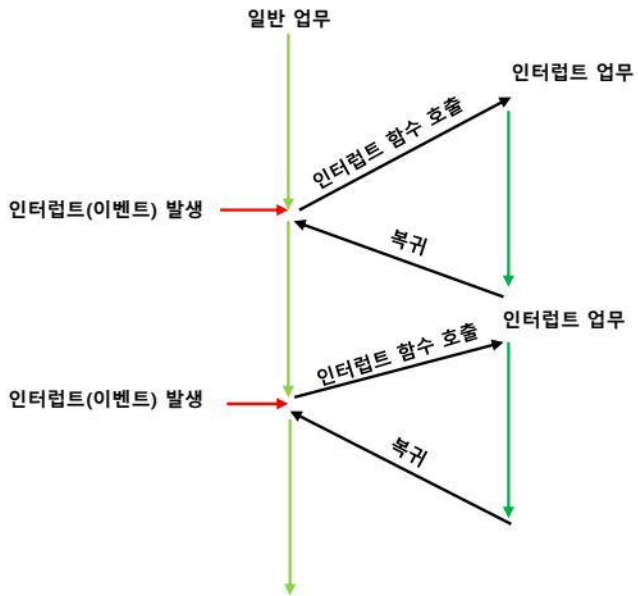
③ 깨어남 확인

```
[kali@kali:04_1]$ps f
    PID TTY      STAT   TIME COMMAND
   1268 pts/1    Ss     0:00 /usr/bin/bash
   1451 pts/1    S+     0:00  \_ sleep 100
   1139 pts/0    Ss     0:00 /usr/bin/bash
   1452 pts/0    R+     0:00  \_ ps f
[kali@kali:04_1]$kill -14 1451
[kali@kali:04_1]$
```

② sleep 프로세스에 14번 시그널 전송

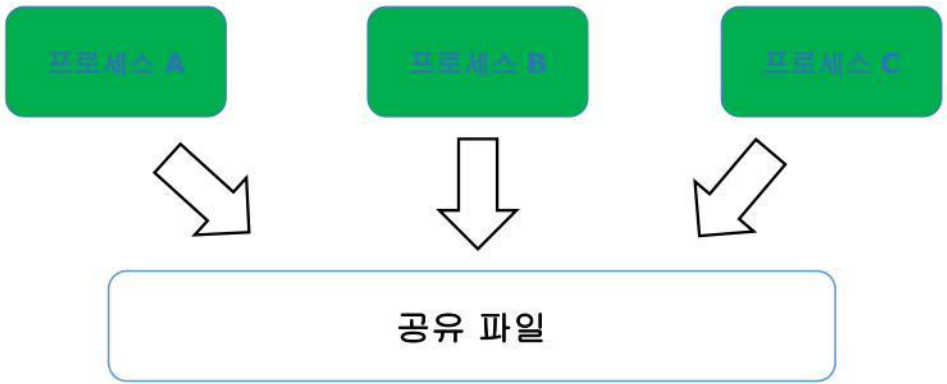
IPC 기법 (3/9)

- **인터럽트**
  - 이벤트 발생에 따른 처리를 위한 방식
  - 프로세스가 동작 중 인터럽트가 발생하면 해당 기능을 수행



IPC 기법 (4/9)

- **공유 파일**
  - 서로 다른 프로세스가 동일한 파일을 공유하여 사용하는 방식
    - fd = open("file.txt", O_RDWR) : 파일 열기, **파일기술자(fd)**를 리턴
    - write(fd, "TEST", 5) : 파일 쓰기
    - read(fd, buf, 5) : 파일 읽기
    - close(fd) : fd가 가리키는 파일을 닫음
  - 프로세스 처리 성능은 기록 장치의 읽기/쓰기 속도에 영향을 받음



(심화) 파일

- **순차 파일(sequential file)**
  - 아무리 큰 파일이라도 파일 내의 데이터는 개념적으로 **한 줄**로 저장됨

(a) 화면 상태 (b) 저장 상태

그림 5-29 파일의 화면 상태와 저장 상태

그림 5-34 순차적 접근의 예

그림 5-30 순차적 접근의 예: 카세트테이프

(심화) 파일

- **파일 기술자**
  - open( ) 함수로 파일을 열면 파일 기술자 fd를 얻음
  - 파일 기술자는 파일 접근 권한 외에 현재 파일의 어느 위치를 읽고 있는지에 대한 정보도 보관
  - 파일에서 파일 기술자는 단 하나
  - 처음 파일이 열리면 파일 기술자는 맨 앞에 위치
  - 파일을 읽거나 쓰면 파일 기술자는 계속 전진

```c
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

void main()
{   int fd;
    char buf[5];

    fd=open("com.txt", O_RDWR);
    read(fd, buf, 5);
    printf("%s", buf);
    close(fd);
    return 0;
}
```

그림 5-31 파일 입출력 코드

(심화) 파일

- **파일 기술자**

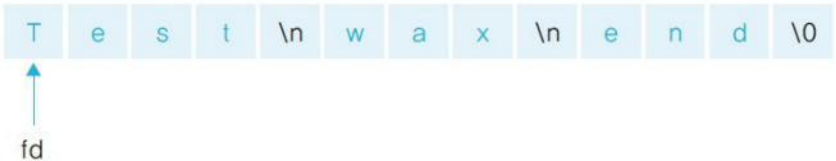


그림 5-32 파일을 처음 열었을 때 파일 기술자의 위치

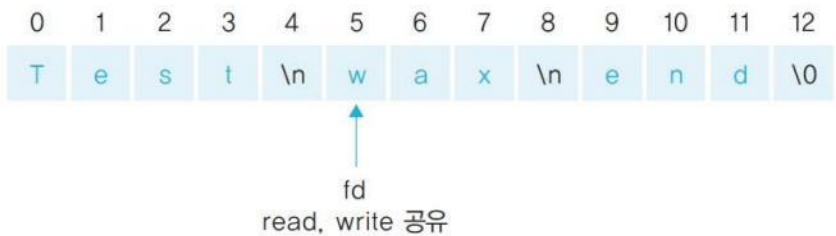
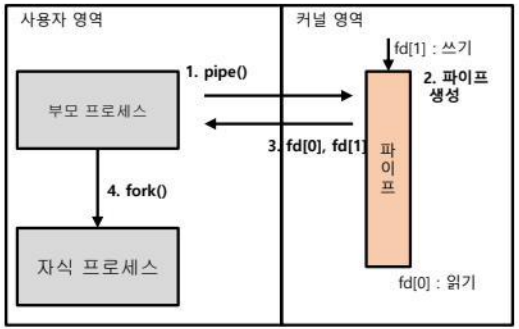
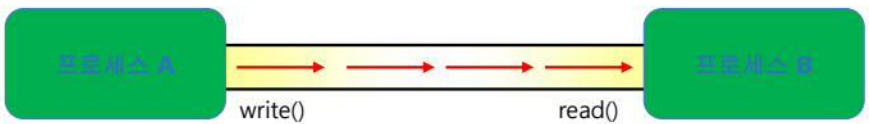


그림 5-33 read 연산 후 파일 기술자의 위치

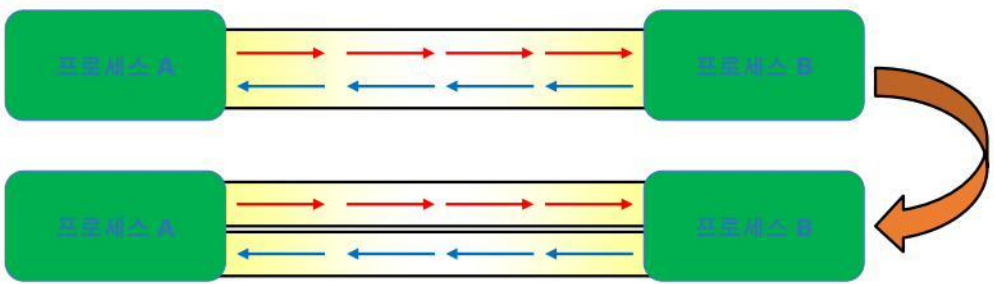
IPC 기법 (5/9)

- **파이프**
  - A 프로세스의 출력을 B프로세스의 입력으로 처리 (ex) ls | grep
  - 파이프는 fork 함수에 의해 복사되지 않음
    - pipe(int fd[ ])





<단방향>



<양방향>

<문제발생>
파이프 안의 값은 주인이 없음
(A가 write 한 값을 B가 안 가져 가면 A가 다시 read하는 문제 발생하므로 파이프 2개 사용)

15

(심화) 파이프

- **파이프를 이용한 통신**
  - 파이프는 파일 기술자를 2개의 원소를 가진 배열로 정의
  - 원소 하나는 읽기용, 하나는 쓰기용으로 사용

```c
#include <stdio.h>
#include <unistd.h>

void main()
{  int pid, fd[2];
   char buf[5];

   if(pipe(fd)==-1) exit(-1);   /* init */
   pid=fork();

   if(pid<0) exit(-1);

   else if(pid==0) {            /* child */
            close(fd[0]);       /* unused */
            write(fd[1], "Test", 5);
            close(fd[1]);
            exit(0); }

   else {                       /* parent */
            close(fd[1]);       /* unused */
            read(fd[0], buf, 5);
            close(fd[0]);
            printf("%s",buf);
            exit(0); }
}
```

```c
#include <stdio.h>
#include <unistd.h>

void main()
{  int pid, fd[2];
   char buf[5];

   if(pipe(fd)==-1) exit(-1);   /* init */
   pid=fork();

   if(pid<0) exit(-1);

   else if(pid==0) {            /* child */
            close(fd[0]);       /* unused */
            write(fd[1], "Test", 5);
            close(fd[1]);
            exit(0); }

   else {                       /* parent */
            close(fd[1]);       /* unused */
            read(fd[0], buf, 5);
            close(fd[0]);
            printf("%s",buf);
            exit(0); }
}
```

그림 5-35 파이프를 이용한 통신 코드

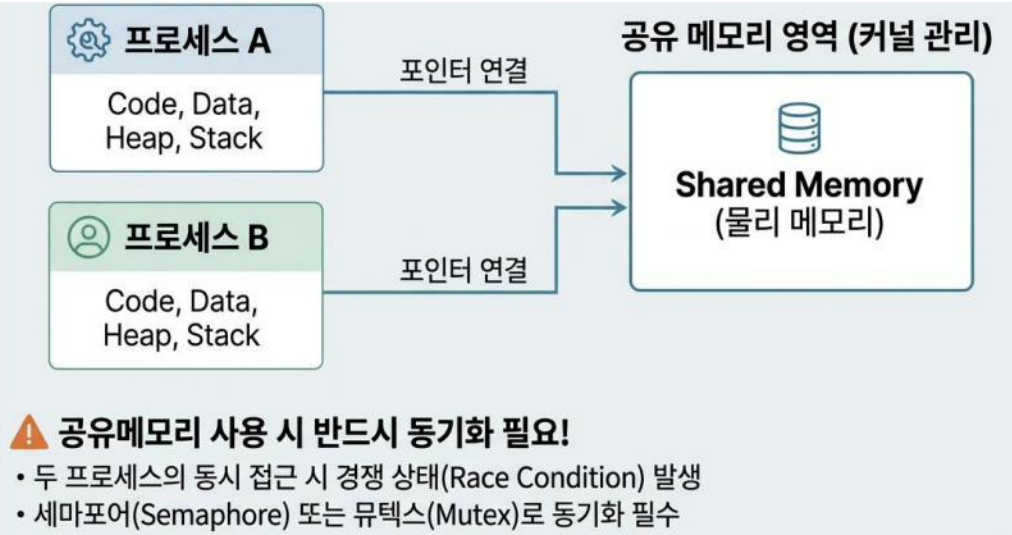
자신이 write 한 데이터를
자신이 read 할 수 있으므로,
반드시 close 해야함

IPC 기법 (6/9)

- **메시지큐**
  - 큐(Queue) 방식으로 데이터를 접근
  - 큐는 행위에 따라 En-queue와 De-queue로 분류
    - En-queue는 큐에 데이터를 삽입
    - De-Queue는 큐에서 데이터를 추출
  - 양방향, 비동기 방식
  - **부모-자식 간의 프로세스가 아니더라도** 프로세스 간 메시지 송수신 가능

IPC 기법 (7/9)

- **공유메모리**
  - 운영체제에서 별도로 할당해준 공유 영역을 활용
  - 각 응용프로그램을 해당 영역을 포인터로 접근
  - 공유메모리 확인 명령: ipcs



IPC 기법 (8/9)

- **공유메모리 실습**
  - 관련 명령어: ipcs, ipcmk, ipcrm
    - ipcs : IPC관련 운영체제내 상태를 확인
    - ipcmk : IPC관련 자원 생성
    - ipcrm : IPC관련 자원 삭제

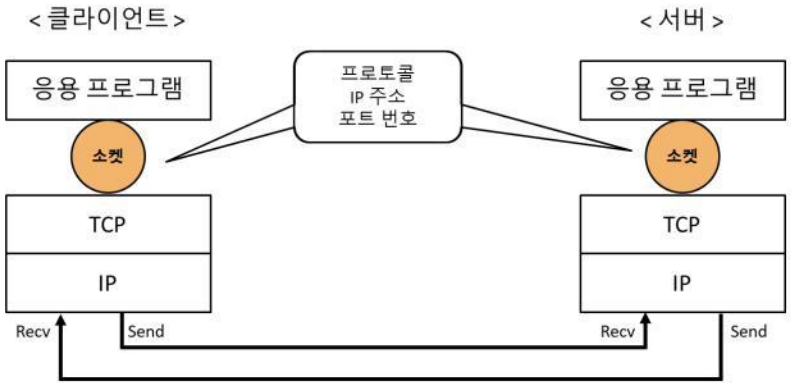
```
[kali@kali:04_1]$ipcmk -M 1024
Shared memory id: 32768
[kali@kali:04_1]$ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x323b47aa 32768      kali       644        1024       0
0x00000000 14         kali       600        524288     2          dest
0x00000000 15         kali       600        2097152    2          dest
0x00000000 16         kali       600        524288     2          dest
```

```
[kali@kali:04_1]$ipcrm -m 32768
[kali@kali:04_1]$ipcs -m

------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x00000000 14         kali       600        524288     2          dest
0x00000000 15         kali       600        2097152    2          dest
0x00000000 16         kali       600        524288     2          dest
0x00000000 22         kali       600        524288     2          dest
0x00000000 24         kali       600        524288     2          dest
```

```
[kali@kali:04_1]$ipcs -mp

------ Shared Memory Creator/Last-op PIDs --------
shmid      owner      cpid       lpid
14         kali       956        573
15         kali       956        573
16         kali       947        1262
22         kali       959        573
24         kali       964        573
30         kali       965        573
33         kali       966        573
37         kali       1003       573
39         kali       963        573
```

IPC 기법 (9/9)

- **소켓**
  - 여러 컴퓨터에 있는 프로세스끼리 통신하는 방법
  - 네트워크 소켓은 컴퓨터 네트워크를 공유하는 프로세스 간의 통신
  - 소켓의 종류(socket()의 옵션)
    - PF_LOCAL : 같은 호스트 내 프로세스 간 통신
    - PF_INET : 인터넷을 통한 다른 호스트 통신(ipv4)
    - PF_INET6 : 인터넷을 통한 다른 호스트 통신(ipv6)



file = open("test_file", ...) read(file) write(file)	=	sock = socket(...생략...) recv(sock) send(sock)

(심화) 네트워킹

- **소켓을 이용한 네트워킹**

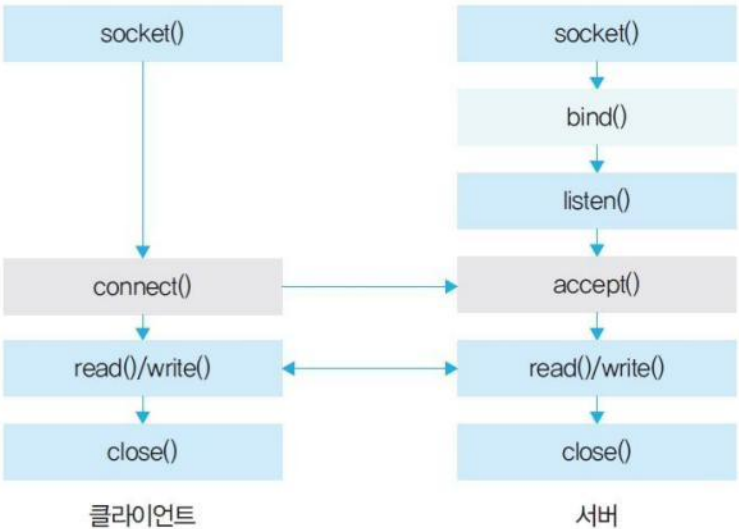


그림 5-38 클라이언트/서버의 통신 절차

임계구역

공유 자원에 대한 접근

- **공유 자원의 접근 예**
  - 공유자원 : 여러 프로세스(또는 스레드)가 동시에 사용하려는 자원

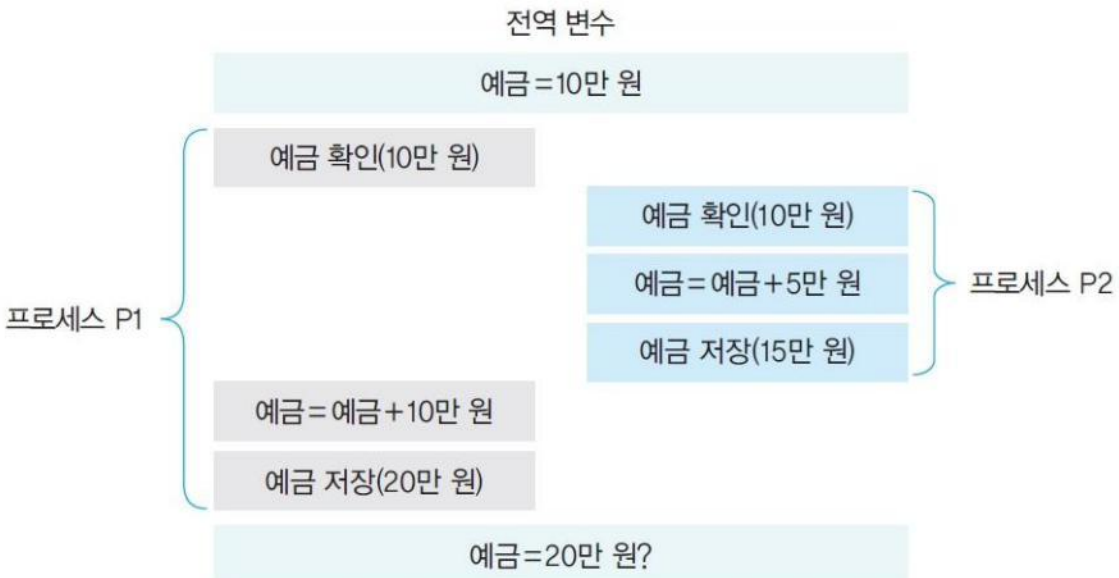


그림 5-6 공유 자원에 대한 접근

임계 구역

- **임계 구역(critical section)**
  - 공유 자원 접근 순서에 따라 실행 결과가 달라지는 프로그램의 영역
  - 믹서는 **공유가 불가능한 자원**으로서 주방의 임계구역
  - 임계구역에서는 프로세스를 **동시에 작업하면 안 됨**
  - 어떤 프로세스가 임계구역에 들어가면 다른 프로세스는 임계구역 밖에서 기다려야 하며 임계구역에 들어간 프로세스가 나와야 들어갈 수 있음

그림 5-7 가스레인지와 믹서

임계구역 문제 해결 조건

- **상호 배제(mutual exclusion)**
  - 한 프로세스가 임계구역에 들어가면 다른 프로세스는 임계구역에 들어갈 수 없음 ➔ (위반시) 데이터 손상
  - 화장실 문 잠금

- **한정 대기(bounded waiting)**
  - 어떤 프로세스도 무한 대기(infinite postpone)하지 않아야 함 ➔ (위반시) 기아상태
  - 언젠가는 반드시 내 차례가 와야 함

- **진행의 융통성(progress flexibility)**
  - 한 프로세스가 다른 프로세스의 진행을 방해해서는 안 됨 ➔ (위반시) 불필요한 대기
  - 빈 화장실은 누구든 들어갈 수 있어야 함

(임계구역 해결위한) 기본 코드 소개

- **임계구역 해결 방법을 설명하기 위한 기본 코드**

```
#include <stdio.h>

typedef enum {false, true} boolean;
extern boolean lock=false;
extern int balance;

main() {
    while(lock==true);
    lock=true;
    balance=balance+10;   /* 임계구역 */
    lock=false;
}
```

그림 5-11 기본 코드

임계구역 해결 조건을 고려한 코드 설계

- **전역 변수로 잠금을 구현한 코드**

```
boolean lock=false;
```
공유 변수

```
while(lock==true);
lock=true;
        임계구역
lock=false;
```
프로세스 P1

```
while(lock==true);
lock=true;
        임계구역
lock=false;
```
프로세스 P2

그림 5-12 전역 변수로 잠금을 구현한 코드

임계구역 해결 조건을 고려한 코드 설계

- **전역 변수로 잠금을 구현한 코드의 문제**

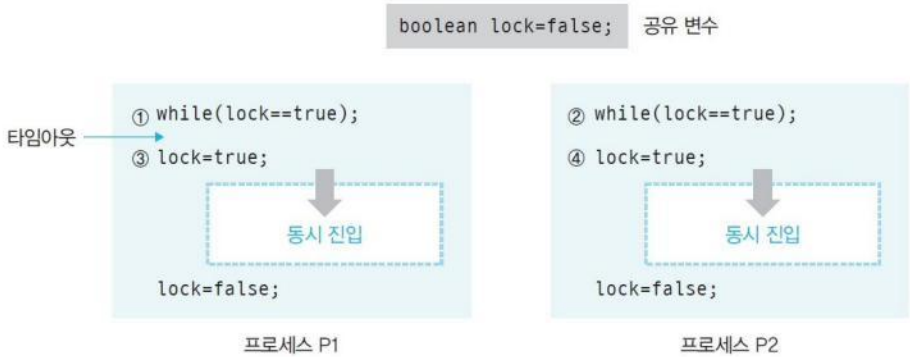


그림 5-13 동시 진입 상황 – 상호 배제 조건을 충족하지 않는 경우

① 프로세스 P1은 while(lock==true); 문 실행

② 프로세스 P2는 while(lock==true); 문 실행

③ 프로세스 P1이 lock=true; 문 실행하여 임계구역에 잠금을 걸고 진입

④ 프로세스 P2도 lock=true; 문 실행하여 임계구역에 잠금을 걸고 진입. 결국 둘 다 임계 구역에 진입 → 상호배제 위반

임계구역 해결 조건을 고려한 코드 설계

- **상호 배제 조건을 충족하는 코드**

```
boolean lock1=false;
boolean lock2=false;
```

공유 변수

```
lock1=true;
while(lock2==true);
        임계구역
lock1=false;
```

프로세스 P1

```
lock2=true;
while(lock1==true);
        임계구역
lock2=false;
```

프로세스 P2

그림 5-14 상호 배제 조건을 충족하는 코드

임계구역 해결 조건을 고려한 코드 설계

- **상호 배제 조건을 충족하는 코드의 문제**

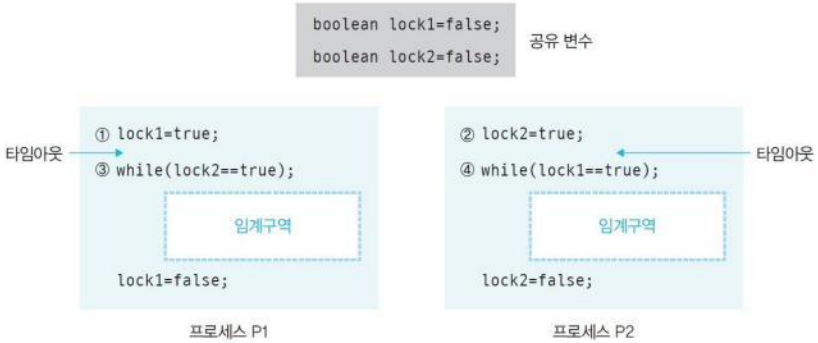


그림 5-15 무한 대기 상황 – 한정 대기 조건을 충족하지 않는 경우

- 프로세스 P1이 lock1=true; 문 실행 후 자신의 CPU 시간 다 씀(타임아웃). 문맥 교환 발생, 프로세스 P2가 실행 상태로 바뀜
- 프로세스 P2도 lock2=true; 문 실행 후 자신의 CPU 시간 다 씀(타임아웃). 문맥 교환 발생, 프로세스 P1이 실행 상태로 바뀜
- 프로세스 P2가 lock2=true; 문 실행하여 프로세스 P1은 while(lock2==true); 문에서 무한 루프에 빠짐
- 프로세스 P1이 lock1=true; 문 실행하여 프로세스 P2도 while(lock1==true); 문에서 무한 루프에 빠짐 → 한정대기 위반

임계구역 해결 조건을 고려한 코드 설계

- **상호 배제와 한정 대기 조건을 충족하는 코드**

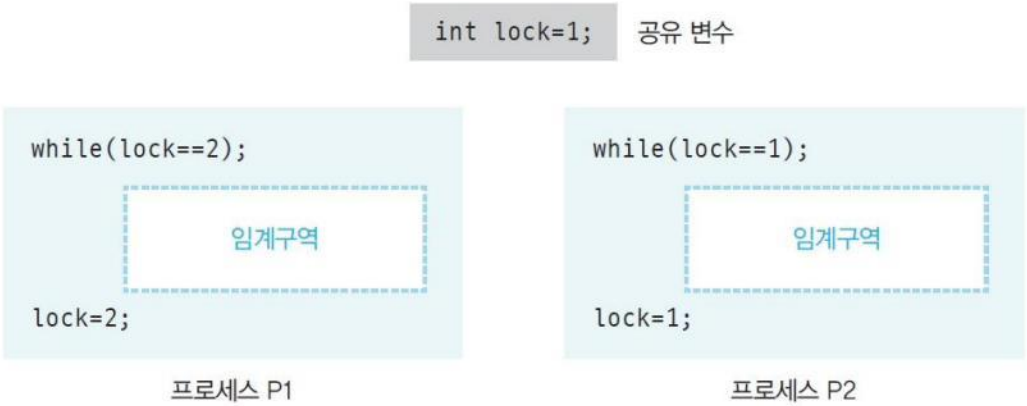


그림 5-16 상호 배제와 한정 대기 조건을 충족하는 코드 – 진행의 융통성 조건을 충족하지 않는 경우

- 반드시 P1과 P2가 번갈아 가면서 실행해야 함
- 만약 P1이 임계구역에 관심 없다면, P2는 영원히 진행 불가 → 진행의 융통성 위반

임계구역 해결 조건을 고려한 코드 설계

- **임계구역 문제의 하드웨어적인 해결 방법**
  - 검사와 지정(test-and-set) 코드로 하드웨어의 지원을 받아 while(lock==true); 문과 lock=true; 문을 한꺼번에 실행
  - 검사와 지정 코드를 이용하면 명령어 실행 중간에 타임아웃이 걸려 임계구역을 보호하지 못하는 문제가 발생하지 않음

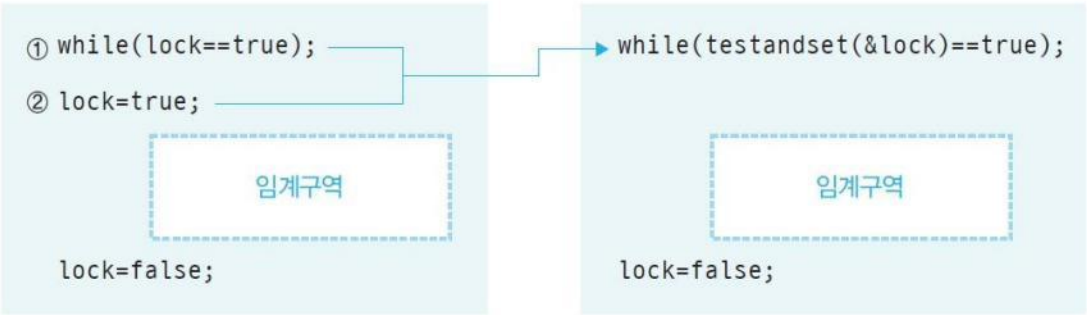


그림 5-17 검사와 지정을 이용한 코드

피터슨 알고리즘

- **피터슨 알고리즘**
  - 두 프로세스 간 임계구역 문제를 소프트웨어적으로 해결 (변수 2개 사용(turn, flag))
  - 임계구역 문제 해결의 세 가지 조건을 모두 만족
  - 2개의 프로세스만 사용 가능하다는 한계가 있음

```
boolean lock1=false;
boolean lock2=false;
int turn=1;
```

공유 변수

```
lock1=true;
turn=2;
While(lock2==true && turn==2);
        임계구역
lock1=false;
```

프로세스 P1

```
lock2=true;
turn=1;
While(lock1==true && turn==1);
        임계구역
lock2=false;
```

프로세스 P2

그림 5-18 피터슨 알고리즘

데커 알고리즘

- **데커 알고리즘**

```
boolean lock1=false;
boolean lock2=false;
int turn=1;
```

공유 변수

```
lock1=true;
while(lock2==true)
{  if(turn==2)  {
        lock1=false;
        while(turn==2);
        lock1=true;  }   /* end if */
}   /* end while */
        임계구역
turn=2;
lock1=false;
```

프로세스 P1

```
lock2=true;
while(lock1==true)
{  if(turn==1)  {
        lock2=false;
        while(turn==1);
        lock2=true;  }   /* end if */
}   /* end while */
        임계구역
turn=1;
lock2=false;
```

프로세스 P2

그림 5-19 데커 알고리즘

데커 알고리즘

- **데커 알고리즘의 동작**

① 프로세스 P1은 우선 잠금을 건다(lock1=true;).

② 프로세스 P2의 잠금이 걸렸는지 확인한다[while(lock2==true)].

③ 만약 프로세스 P2도 잠금을 걸었다면 누가 먼저인지 확인한다[if(turn==2)]. 만약 프로세스 P1의 차례라면(turn==1) 임계구역으로 진입하고 프로세스 P2의 차례라면(turn==2) ④로 이동

④ 프로세스 P1은 잠금을 풀고(lock1=false;) 프로세스 P2가 작업을 마칠 때까지 기다린다 [while(turn==2);]. 프로세스 P2가 작업을 마치면 잠금을 걸고(lock1=true;) 임계구역으로 진입

세마포어

- **세마포어(semaphore)**
  - 프로세스가 임계구역에 진입하기 전에 스위치를 사용 중으로 놓음
  - 후에 도착하는 프로세스는 앞의 프로세스가 작업을 마칠 때까지 기다림
  - 프로세스가 작업을 마치면 세마포어가 다음 프로세스에 임계구역을 사용하라는 동기화 신호를 보냄

그림 5-20 세마포어와 토글 스위치

```
Semaphore(n);
P();
        임계구역
V();
```

그림 5-21 세마포어 코드

세마포어

- **세마포어 내부 코드**
  - **Semaphore(n)** : 전역 변수 RS를 n으로 초기화, RS에는 현재 사용 가능한 자원의 수 저장
  - **P()** : 잠금을 수행하는 코드로 RS가 0보다 크면(사용 가능한 자원이 있으면) 1만큼 감소시키고 임계구역에 진입, RS가 0보다 작으면(사용 가능한 자원이 없으면) 0보다 커질 때까지 기다림
  - **V()** : 잠금 해제와 동기화를 같이 수행하는 코드로, RS 값을 1 증가시키고 세마포어에서 기다리는 프로세스에게 임계구역에 진입해도 좋다는 wake_up 신호를 보냄

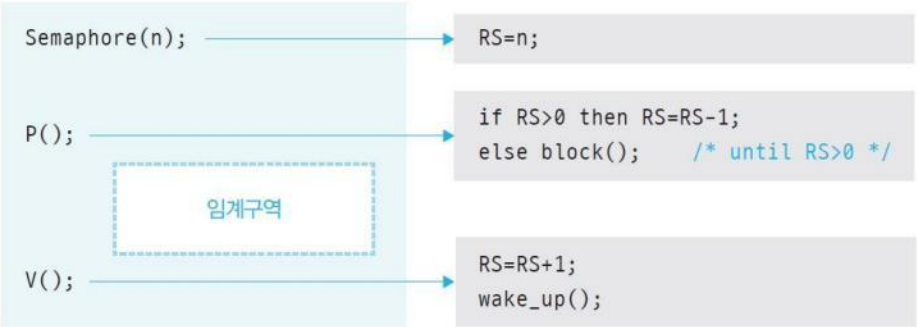


그림 5-22 세마포어 내부 코드

세마포어

- **세마포어를 사용해 예금 5만 원이 사라진 문제를 해결한 코드**

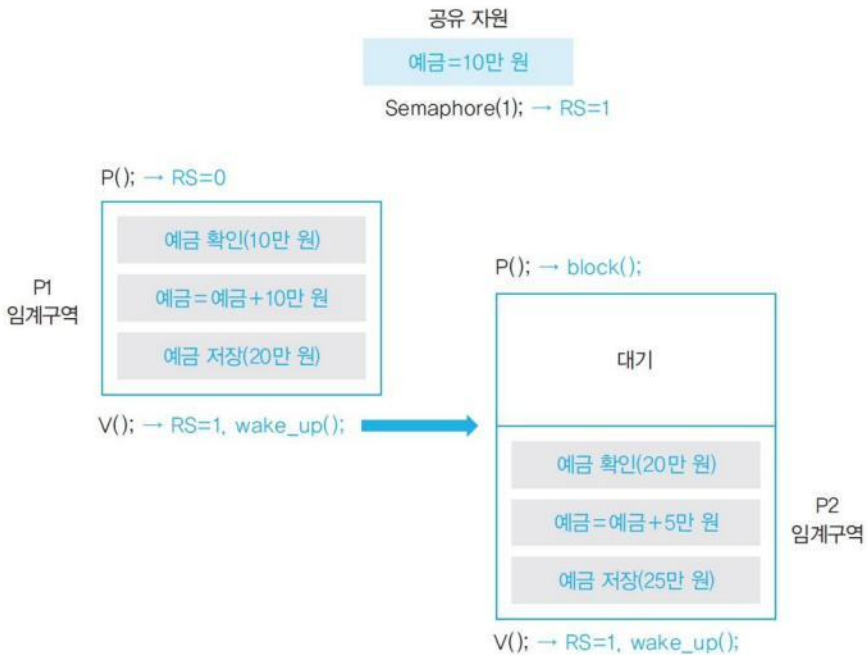


그림 5-23 세마포어 사용 예

세마포어

- **공유 자원이 여러 개일 때 세마포어 사용 예**

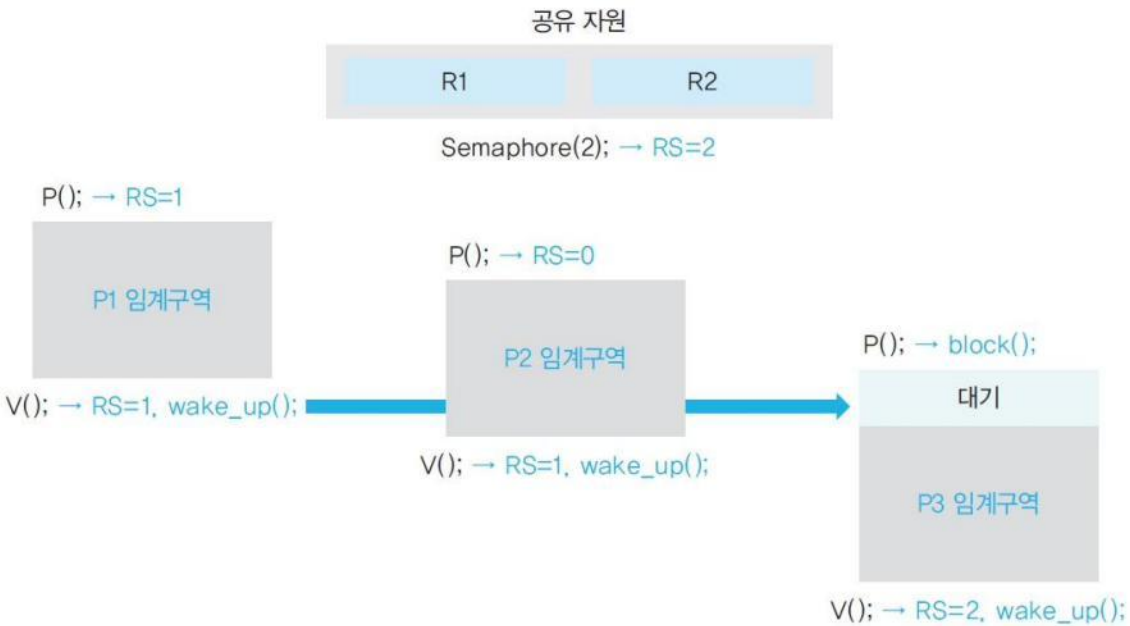


그림 5-24 공유 자원이 여러 개일 때의 세마포어 사용 예

세마포어

- **세마포어의 잘못된 사용 예**

그림 5-25 세마포어의 잘못된 사용 예

① 프로세스가 세마포어 사용하지 않고 임계구역에 바로 들어간 경우로, 임계구역을 보호할 수 없음

② P()를 두 번 사용하여 wake_up 신호가 발생하지 않은 경우로, 프로세스 간 동기화가 이루어지지 않아 세마포어 큐에서 대기하던 프로세스들이 무한 대기에 빠짐

③ P()와 V()를 반대로 사용해 상호 배제가 보장되지 않은 경우로, 임계구역을 보호할 수 없음

모니터

- **모니터(monitor)**
  - 공유 자원을 내부적으로 숨기고 공유 자원에 접근하기 위한 인터페이스만 제공해 자원을 보호하고 프로세스 간의 동기화를 시킴
  - 자원을 숨기고 인터페이스만 제공하는 시스템 호출과 같은 개념
- **모니터의 작동 원리**

① 임계구역으로 지정된 변수나 자원에 접근하려는 프로세스는 직접 P()나 V()를 사용하지 않고 모니터에 작업 요청

② 모니터는 요청받은 작업을 모니터 큐에 저장 후 순서대로 처리하고 결과만 해당 프로세스에 알려줌

그림 5-26 모니터의 작동 원리

모니터

- 예금 5만 원이 사라진 문제를 모니터 사용해 해결한 코드

- 자바로 작성한 모니터 내부 코드 일부

P1	increase(10);	P2	increase(5);

그림 5-27 모니터 사용법

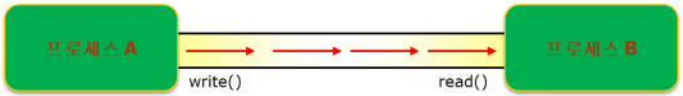
```
monitor shared_balance {
   private:
      int balance=10;          /* shared data */
      boolean busy=false;
      condition mon;           /* condition variable */

   public:
      increase(int amount) {           /* public interface */
          if(busy==true) mon.wait();   /* waiting in queue */
          busy=true;
          balance=balance+amount;
          mon.signal();        /* wake up next waiting process */
      }
   …
```

그림 5-28 자바로 작성한 모니터 내부 코드(일부)

[실습] 파이프(단방향)

열기(O) pipe1.c ~/ 저장(S)

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFSIZE 30

int main(int argc, int* argv[])
{
        int fd[2];
        char buffer[BUFSIZE];
        pid_t pid;
        int state;

        state = pipe(fd);
        if(state == -1)
        {
                puts("pipe() error");
                exit(1);
        }

        pid = fork();

        if(pid == 0)
        {
                write(fd[1], "pipe connection success\n", 25);
        }
        else
        {
                read(fd[0], buffer, BUFSIZE);
                puts(buffer);
        }

        return 0;
}
```

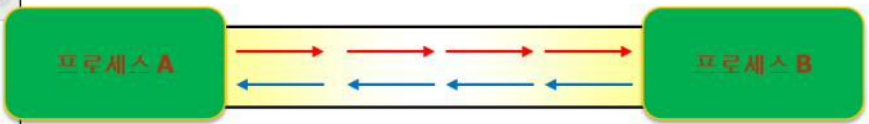



```
comso@comso-VirtualBox:~$ gcc pipe1.c -o pipe1
comso@comso-VirtualBox:~$ ./pipe1
pipe connection success
```

[실습] 파이프(양방향)

열기(O) pipe2.c 저장(S)

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFSIZE 30

int main(int argc, int* argv[])
{
        int fd[2];
        char buffer[BUFSIZE];
        pid_t pid;
        int state;

        state = pipe(fd);
        if(state == -1)
        {
                puts("pipe() error");
                exit(1);
        }

        pid = fork();

        if(pid == 0)
        {
                write(fd[1], "child send message\n", 20);
                sleep(1);
                read(fd[0], buffer, BUFSIZE);
                printf("Child receive: %s\n", buffer);
        }
        else
        {
                read(fd[0], buffer, BUFSIZE);
                printf("Parent Process receive: %s\n", buffer);
                write(fd[1], "parent message\n", 16);
                sleep(1);
        }

        return 0;
}
```




```
comso@comso-VirtualBox:~$ gcc pipe2.c -o pipe2
comso@comso-VirtualBox:~$ ./pipe2
Parent Process : child send message

comso@comso-VirtualBox:~$ Child Process : parent send message
```

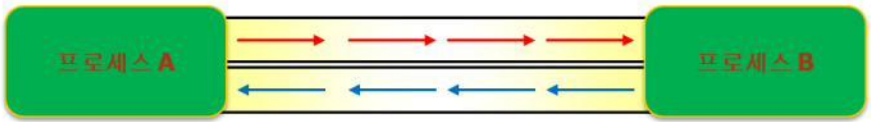
※ 예시 코드에서 sleep(1)을 삭제하면?

```
comso@comso-VirtualBox:~$ ./pipe2
Child receive: child send message
```

[실습] 파이프(양방향)

열기(O) pipe3.c ~/ 저장(S)

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFSIZE 30

int main(int argc, int* argv[])
{
        int fd1[2], fd2[2];
        char buffer[BUFSIZE];
        pid_t pid;
        int state;

        if(pipe(fd1) == -1 || pipe(fd2) == -1)
        {
                puts("pipe() error");
                exit(1);
        }

        pid = fork();

        if(pid == 0)
        {
                write(fd1[1], "child send message\n", 20);
                read(fd2[0], buffer, BUFSIZE);
                printf("child process recieve : %s\n", buffer);
        }
        else
        {
                read(fd1[0], buffer, BUFSIZE);
                printf("parent process recieve : %s\n", buffer);
                write(fd2[1], "parent message\n", 16);
                sleep(1);
        }

        return 0;
}
```




```
comso@comso-VirtualBox:~$ gcc pipe3.c -o pipe3
comso@comso-VirtualBox:~$ ./pipe3
parent process recieve : child send message

child process recieve : parent message
```